

-- { БАЗОВЫЙ УРОВЕНЬ - ОСНОВЫ PYTHON } --

Ethical Hacking Tutorial Group The Codeby

представляет



Основы программирования на

PYTHON



-- { для начинающих } --

Оглавление

Список – List	2
Методы списков	4
Метод insert	4
Методы append и extend	4
Метод sort	5
Метод reverse	6
Метод pop	6
Метод remove и инструкция del	7
Метод count	7
Функции max и min	8
Функция sum	8
Преобразование списков в строку и обратно	10
Как преобразовать список в строку?	10
Как преобразовать строку в список?	10

Список – List

С самой простой последовательностью – строками – мы познакомились на прошлом уроке, а на этом – изучим более сложный тип данных Списки.

Списки – это упорядоченные изменяемые коллекции объектов произвольных типов. В Python списки очень похожи на массивы в других языках программирования. Списки можно создать, перечислив объекты через запятую в квадратных скобках, или используя встроенную функцию `list`, применив её, например, к строке, но уже с круглыми скобками.

```
main_list_one = ['Мир', 5, "Дружба", "Жевачка", 3.14]
main_list_two = list('Мир')
print(main_list_one, main_list_two)
```

```
1 x
['Мир', 5, 'Дружба', 'Жевачка', 3.14] ['М', 'и', 'р']
```

Обратите внимание – передав в функцию `list` строку, мы получили список букв – строк единичной длины. Если бы нам нужно было создать список из одного слова, нужно было использовать двойные скобки и запятую: `list(('Мир',))`

Списки также могут включать в себя другие списки – список списков.

```
main_list_one = [[1, 2, 3], [4, 5, 6]]
print(main_list_one)
```

```
1 x
[[1, 2, 3], [4, 5, 6]]
```

Вложенные списки представляют собой следующий вид:

```
main_list_one = [[1, 2, 3], [4, 5, 6]]
print(len(main_list_one))
print(type(main_list_one))
```

```
1 x
2
<class 'list'>
```

То есть внутри одного списка располагается один или более списков

К спискам можно применять самые методы, подходящие для итерируемых объектов: взятие по индексу, срезы, циклы и т.д.

Итерируемый объект (iterable) — это объект, который способен возвращать элементы по одному.

Итерация — это последовательность инструкций, которая повторяется определенное количество раз, или до выполнения указанного условия.

Итерирование по своим элементам поддерживается у таких типов данных как списки, строки, кортежи, словари, файлы.

Подробнее про итерацию мы будем говорить в теме Циклы.

Пример с циклом (циклы будем изучать позже).

```
main_list = [1, 2, 3, 4, 5]
for i in range(len(main_list)):
    main_list[i] **= 2
print(main_list)
```

```
1 x
[1, 4, 9, 16, 25]
```

Обратите внимание — каждый элемент списка может быть другим типом данных и даже другим списком.

```
main_list = ['Горох', 'Морковь', 'Свекла', [4, 5, 6]]
print(main_list[2])
print(type(main_list[2]))
print(main_list[-1])
print(type(main_list[-1]))
print(main_list[1:-1])
```

```
1 x
Свекла
<class 'str'>
[4, 5, 6]
<class 'list'>
['Морковь', 'Свекла']
```

Методы списков

Метод insert

Метод **insert** вставляет переданный аргумент в определённое место в списке. Он принимает два аргумента: номер индекса и сам объект – новый элемент списка.

```
main_list = [1, 2, 3, 4, 5]
main_list.insert(2, 'String')
print(main_list)
```

```
1 x
[1, 2, 'String', 3, 4, 5]
```

Методы append и extend

Метод **append** в отличие от insert добавляет элемент в конец списка. Метод **extend** умеет добавлять в конец списка сразу несколько элементов. Когда подаётся строка этим методом, её символы перебираются и добавляются в конец списка.

```
main_list = [1, 2, 3, 4, 5]
main_list.append('String')
print(main_list)
main_list.extend('String')
print(main_list)
```

```
1 x
[1, 2, 3, 4, 5, 'String']
[1, 2, 3, 4, 5, 'String', 'S', 't', 'r', 'i', 'n', 'g']
```

Переданный объект вставляется внутрь «как есть», не разбиваясь на отдельные элементы. В то время как extend распакует их и вставит по очереди в список.

```
main_list = [1, 2, 3, 4, 5]
new_list = ['firefox', 'tor', 'chrome']
main_list.append(new_list)
print(main_list)
```

```
1 x
[1, 2, 3, 4, 5, ['firefox', 'tor', 'chrome']]
```

```
main_list = [1, 2, 3, 4, 5]
new_list = ['firefox', 'tor', 'chrome']
main_list.extend(new_list)
print(main_list)
```

```
1 x
[1, 2, 3, 4, 5, 'firefox', 'tor', 'chrome']
```

Метод sort

Метод `sort` применяется для сортировки элементов списка. По умолчанию все элементы выводятся в порядке возрастания. Метод может принимать необязательные аргументы `key` и `reverse`.

```
new_list = ['firefox', '5', 'tor', '0', 'chrome', '500']
new_list.sort()
print(new_list)
```

```
1 x
['0', '5', '500', 'chrome', 'firefox', 'tor']
```

Если в качестве аргумента указать `reverse=True`, список будет отсортирован в порядке убывания.

```
new_list = ['firefox', '5', 'tor', '0', 'chrome', '500']
new_list.sort(reverse = True)
print(new_list)
```

```
1 x
['tor', 'firefox', 'chrome', '500', '5', '0']
```

Для смешанных типов данных в списке, этот метод не годится.

```
new_list = ['firefox', 5, 'tor', 0, 'chrome', 500]
new_list.sort()
print(new_list)
```

```
1 x
Traceback (most recent call last):
  File "/root/PycharmProjects/education/1.py", line 2, in <module>
    new_list.sort()
TypeError: '<' not supported between instances of 'int' and 'str'
```

Аргумент `key` в качестве параметра принимает функцию.

Отсортируем список по длине строки.

```
new_list = ['firefox', '5', 'tor', '0', 'chrome', '500']
new_list.sort(key=len)
print(new_list)
```

```
1 x
['5', '0', 'tor', '500', 'chrome', 'firefox']
```


Или по целым числам.

```
new_list = [56, 5, 6546, 0, 9, 500]
new_list.sort(key=int)
print(new_list)
```

1 ×

```
[0, 5, 9, 56, 500, 6546]
```

Метод reverse

Изменить порядок элементов в списке задом наперёд позволяет метод **reverse**.

```
new_list = ['Дуб', 'Бук', 'Орех', 'Сосна', 'Ель', 'Ясень']
new_list.reverse()
print(new_list)
```

1 ×

```
['Ясень', 'Ель', 'Сосна', 'Орех', 'Бук', 'Дуб']
```

Обратите внимание – метод ничего не возвращает, он изменяет тот объект у которого вызван.

Метод pop

Удалить любой элемент списка можно с помощью метода **pop**. Удаление элемента идёт по индексу, а если он не указан, удаляется последний элемент списка. Индекс указывается без квадратных скобок.

```
new_list = ['Дуб', 'Бук', 'Орех', 'Сосна', 'Ель', 'Ясень']
new_list.pop(2)
print(new_list)
new_list.pop()
print(new_list)
```

1 ×

```
['Дуб', 'Бук', 'Сосна', 'Ель', 'Ясень']
['Дуб', 'Бук', 'Сосна', 'Ель']
```

Метод remove и инструкция del

Удалить элемент можно методом `remove`, указав подлежащий удалению элемент в качестве параметра. Инструкция `del` имеет точно такое же действие, как и для переменных – удаляет элемент по индексу, который указывается в квадратных скобках. Инструкция `del` пишется отдельно, а не через точку, как метод.

```
new_list = ['Дуб', 'Бук', 'Орех', 'Сосна', 'Ель', 'Ясень']
new_list.remove('Бук')
print(new_list)
del new_list[-2]
print(new_list)
```

```
1 x
['Дуб', 'Орех', 'Сосна', 'Ель', 'Ясень']
['Дуб', 'Орех', 'Сосна', 'Ясень']
```

Инструкция `del` поддерживает так же и срезы.

```
new_list = ['Дуб', 'Бук', 'Орех', 'Сосна', 'Ель', 'Ясень']
del new_list[1:-2]
print(new_list)
```

```
1 x
['Дуб', 'Ель', 'Ясень']
```

Метод count

Количество вхождений элемента в список можно узнать методом `count`.

```
new_list = [4, 18, -5, 4, 44, 444]
print(new_list.count(4))
```

```
1 x
2
```


Функции max и min

Функция **max** возвращает элемент с максимальным значением, а **min** соответственно с минимальным.

```
num = [7, 58, 568, 1000, 1]
print(max(num))
print(min(num))
```

```
1 x
1000
1
```

```
one_list = ['yes', 'no', 'never', 'always']
print(max(one_list))
print(min(one_list))
```

```
1 x
yes
always
```

В случае, когда элементы строчные, и несколько элементов начинаются с одной буквы, сравниваются последующие символы, чтобы определить элемент с минимальным или максимальным значением.

```
one_list = ['yes', 'y', 'never', 'always', 'a']
print(max(one_list))
print(min(one_list))
```

```
1 x
yes
a
```

Функция sum

Для числовых значений списка можно воспользоваться функцией **sum**.

```
number_list = [4, 56, 89, -56, 23]
print(sum(number_list))
```

```
1 x
116
```

К спискам, как и к строкам можно применять операции конкатенации и умножения.

```
one_list = ['yes', 'no', 'never', 'always']  
two_list = ['aha']  
print(one_list + two_list)  
print(two_list * 5)
```

1 ×

```
['yes', 'no', 'never', 'always', 'aha']  
['aha', 'aha', 'aha', 'aha', 'aha']
```

Создание списка с одинаковыми элементами.

```
num = 5  
list_num = [100] * num  
print(list_num)
```

1 ×

```
[100, 100, 100, 100, 100]
```

Преобразование списков в строку и обратно

Как преобразовать список в строку?

Чтобы получить из списка строки, воспользуемся методом `join`. В кавычках укажем любой желаемый разделитель.

Метод НЕ работает со смешанным типом данных!

```
new_list = ['день', 'тень', 'лень', 'пень']
print(' '.join(new_list))
print(', '.join(new_list))
print('/'.join(new_list))
```

```
1 x
день тень лень пень
день, тень, лень, пень
день/тень/лень/пень
```

Как преобразовать строку в список?

Метод `split` строку преобразовывает в список.

```
main_string = 'выйду в поле вместе с конём'
print(main_string.split())
```

```
1 x
['выйду', 'в', 'поле', 'вместе', 'с', 'конём']
```

Если мы не указываем аргумент в методе, то деление происходит автоматически – по всем пробельным символам. Но мы можем указать необходимую строку для деления в скобках.

```
main_string = 'Маша, Саша, Даша'
print(main_string.split(','))
```

```
1 x
['Маша', ' Саша', ' Даша']
```